CODESPECT

Dutch Protocol

SECURITY ASSESSMENT REPORT

February 4, 2026

Prepared for:

DUTCH

Contents

1 About CODESPECT

CODESPECT is a specialized smart contract security firm dedicated to ensure the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to build secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialize in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralized applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialized cybersecurity solutions designed to minimize risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implementing a bug bounty program to incentivize external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

3.1 Impact

- **High** - Results in a substantial loss of assets (more than 10%) within the protocol or causes significant disruption to the majority of users.
- **Medium** - Losses affect less than 10% globally or impact only a portion of users, but are still considered unacceptable.
- **Low** - Losses may be inconvenient but are manageable, typically involving issues like griefing attacks that can be easily resolved or minor inefficiencies such as gas costs.

3.2 Likelihood

- **High** - Very likely to occur, either easy to exploit or difficult but highly incentivized.
- **Medium** - Likely only under certain conditions or moderately incentivized.
- **Low** - Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.3 Action Required for Severity Levels

- **Critical** - Must be addressed immediately if already deployed.
- **High** - Must be resolved before deployment (or urgently if already deployed).
- **Medium** - It is recommended to fix.
- **Low** - Can be fixed if desired but is not crucial.

In addition to High, Medium, and Low severity levels, CODESPECT utilizes two other categories for findings: **Informational** and **Best Practices**.

a) **Informational** findings do not pose a direct security risk but provide useful information the audit team wants to communicate formally.

b) **Best Practices** findings indicate that certain portions of the code deviate from established smart contract development standards.

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for the smart contracts of DUTCH. DUTCH protocol is an NFT auction platform featuring integrated deflationary tokenomics. The system implements a bonding curve-backed token economy designed to create sustainable value accrual through auction fee monetization and systematic token supply reduction.

This audit focuses on the entire protocol.

The audit was performed using:

a) Manual analysis of the codebase.

b) Dynamic analysis of smart contracts, execution testing.

CODESPECT found twenty-eight points of attention, seven classified as `High`, seven classified as `Medium`, ten classified as `Low` and four classified as `Informational`. All of the issues are summarised in Table 2.

Audit Conclusion

CODESPECT conducted a comprehensive security assessment of the Dutch Protocol smart contracts. During the audit phase, the CODESPECT auditing team identified numerous vulnerabilities, as described in this report, and concluded that the protocol required substantial remediation. The protocol team has since addressed all identified issues, and the fixes have been verified by the auditors. Nevertheless, due to the volume and severity of findings discovered during the initial assessment, CODESPECT recommends that the protocol undergo an additional independent security audit prior to or shortly after deployment.

Organisation of the document is as follows:

- **Section 5** summarizes the audit.
- **Section 6** describes the functionality of the code in scope.
- **Section 7** presents the issues.
- **Section 8** discusses the documentation provided by the client for this audit.
- **Section 9** presents the compilation and tests.

Issues found:

Severity	Unresolved	Fixed	Acknowledged
High	0	7	0
Medium	0	6	1
Low	0	9	1
Informational	0	3	1
Total	**0**	**25**	**3**

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

Audit Type	Security Review
Project Name	Dutch Protocol
Type of Project	NFT Auction Platform
Duration of Engagement	10 Days
Duration of Fix Review Phase	3 Days
Draft Report	December 5, 2025
Final Report	February 4, 2026
Repository	dutch-protocol
Commit (Audit)	cec8b243aa645fdbe80e05e613eca69542b62a2e
Commit (Final)	6034f8f145997c4cd427fbae396330621153c444
Documentation Assessment	High
Test Suite Assessment	Low
Auditors	shaflow01, 0xSynthrax, 0xbountyhunt3r

Table 3: Summary of the Audit

5.1 Scope - Audited Files

	Contract	LoC
1	DutchVault.sol	698
2	DUTCHBondingHook.sol	687
3	DutchAuctionMarketplace.sol	552
4	AuctionAssist.sol	356
5	Presale.sol	302
6	DUTCHToken.sol	137
7	interfaces/IDutchAuctionMarketplace.sol	17
8	interfaces/IAuctionAssist.sol	16
9	interfaces/IDutchVault.sol	16
10	interfaces/IDUTCHToken.sol	8
	Total	**2789**

5.2 Findings Overview

	Finding	Severity	Update
1	Contribution oversubscription will cause `calimReward(...)` call revert due to insufficient funds for late callers	High	Fixed
2	Contributions not reset after purchase allows infinite reward claims	High	Fixed
3	DOS in `_createVestingStreams(...)` due to unbounded loop	High	Fixed
4	Incorrect `SpecifiedDelta` returned	High	Fixed
5	Inverted sign convention for `amountSpecified` breaks swap logic	High	Fixed
6	Tax fee revenue may be lost due to Uniswap V4 trading	High	Fixed
7	`settleWithETH(...)` function call will revert because of ETH refunds	High	Fixed
8	Bonding-curve taxes and burn revenue cannot be distributed within the collection	Medium	Fixed
9	Inventory costETH records max price instead of actual cost causing cascading errors	Medium	Fixed
10	Marketplace refunds misattributed across collections causing contributor reward loss	Medium	Acknowledged
11	Stuck NFTs block rebalancing and write-off workaround causes contributor reward loss	Medium	Fixed
12	The `contribute(...)` function may be DoS	Medium	Fixed
13	The `swapETHForDUTCH` function does not charge royalty fees	Medium	Fixed
14	User's contribution can be rebalanced to other collections	Medium	Fixed
15	Bad ownership check will prevent `DutchVault` from buying NFTs that were purchased in the past	Low	Fixed
16	Failed vesting stream creation causes permanent loss of contributor funds	Low	Fixed
17	Fake NFT contract attack due to missing ownership verification	Low	Fixed
18	Inappropriate slippage control is present in `settleWithETH(...)`	Low	Fixed
19	It's possible to create duplicate listings in `DutchAuctionMarketplace`	Low	Fixed
20	The `presale(...)` function may be DoS	Low	Fixed
21	Unused `vaultFee`	Low	Acknowledged
22	`_buyNFTInternal(...)` function uses unchecked calldata for NFT purchases	Low	Fixed
23	`record.listed` status is not updated after the NFT is sold	Low	Fixed
24	`withdrawCollectionBalance(...)` can withdraw user contributions	Low	Fixed
25	Improper collection configuration checks	Info	Fixed
26	Open TODOs	Info	Fixed
27	The price growth has no upper limit	Info	Acknowledged
28	`_lastBuyBlock` is not fully updated	Info	Fixed

6 System Overview

DUTCH is an NFT auction platform integrated with a deflationary tokenomics model. The system uses DUTCH tokens as the settlement medium for NFT purchases and incorporates a token economy supported by a bonding curve. By capturing value from auction fees and systematically reducing token supply, it enhances token scarcity, thereby promoting the long-term value growth of DUTCH tokens and the sustainable development of its ecosystem.

The DUTCH protocol can be divided in 5 parts:

- **The DUTCH token:** DUTCH is an ERC20 standard token with transfer restrictions and blacklist management. Price discovery is carried out by the bonding-curve engine defined in the hook. This token is used for purchasing and settling NFTs within the system.
- **The Bonding hook:** The Bonding hook is a Uniswap V4 hook that implements a supply-based price curve for the DUTCH token. It replaces the AMM math for swaps in a dedicated WETH/DUTCH V4 pool, enforces a symmetric buy-sell tax, and provides funding for NFT purchases within the protocol.
- **The DutchVault and DutchAuctionMarketplace:** The DutchVault receives ETH fees generated by the Bonding hook and the DutchAuctionMarketplace, as well as ETH contributed by users through AuctionAssist. It uses these ETH to purchase NFTs from specified collections on external authorized marketplaces, which are then listed in the DutchAuctionMarketplace for Dutch auctions.
- **The AuctionAssist:** It allows users to collectively contribute funds to the DutchVault for NFT purchases. When NFTs are sold, they receive DUTCH token rewards proportionally, while the protocol burns any remaining tokens.
- **The Presale:** The Presale is a pre-launch sale of DUTCH tokens before the system goes live. Users contribute ETH, which is used to mint DUTCH tokens via the bonding curve after the presale ends, and the tokens are streamed proportionally to presale participants using Sablier V2.

6.1 DUTCH token trading Flow

DUTCH token trading is restricted by the protocol to occur primarily in a dedicated WETH/DUTCH V4 pool. Major DEXes are blacklisted from trading DUTCH tokens.

The WETH/DUTCH V4 pool does not hold actual liquidity. When users call swap to trade DUTCH, the _beforeSwap function in the Bonding Hook is executed.

```
function _beforeSwap(
    address sender_,
    PoolKey calldata key_,
    SwapParams calldata params_,
    bytes calldata hookData_
)
    internal
    override
    nonReentrant
    whenNotPaused
    returns (bytes4 selector_, BeforeSwapDelta delta_, uint24 fee_)
{
```

This function takes the user input, calculates the output according to the custom bonding curve, and sets BeforeSwapDelta to offset params_.amountSpecified, effectively bypassing the AMM calculation in Uniswap V4.
In afterSwap, the accountDelta generated by the hook is transferred to the sender.

6.2 NFT trading Flow

During DUTCH token trading, the _beforeSwap function in the Bonding Hook collects ETH tax fees.

The DutchAuctionMarketplace charges ETH listing fees when external users list NFTs using the system. These ETH proceeds are sent to the DutchVault and, in the receive() function, are distributed proportionally to each collection according to the predefined _allocations.

```
function contribute(address collection_)
    external
    payable
    nonReentrant
    whenNotPaused
{
```

At the same time, `AuctionAssist` allows external users to call the `contribute` function to contribute ETH for NFT purchases for a specified collection.

```
function buyNFTForCollection(
    address collection_,
    uint256 tokenId_,
    address marketplace_,
    uint256 value_,
    bytes calldata marketplaceCalldata_
) external nonReentrant whenNotPaused {
```

Once a collection has sufficient funds, anyone can call `buyNFTForCollection` to purchase NFTs for that collection using the vault's funds.

```
function listNFTOnMarketplace(address collection_, uint256 tokenId_)
    external
    nonReentrant
    whenNotPaused
{
```

After the NFT is successfully purchased, anyone can call `listNFTOnMarketplace` to list the NFT in the `DutchAuctionMarketplace` for a Dutch auction.

```
function settle(uint256 listingId_, uint256 maxPriceDUTCH_)
    public
    whenNotPaused
    nonReentrant
{
function settleWithETH(uint256 listingId_, uint256 maxPriceETH_)
    external
    payable
    whenNotPaused
    nonReentrant
{
```

Buyers can call `settle` or `settleWithETH` to purchase NFTs listed on the `DutchAuctionMarketplace`. If there are user contributions, the `DUTCH` tokens used for the purchase are distributed to contributors proportionally via `AuctionAssist`, and any remaining `DUTCH` tokens are burned.

6.3 Presale Flow

The presale represents the first minting of `DUTCH` tokens, and specific flow controls are enforced to prevent arbitrageurs from front-running and purchasing `DUTCH` at the curve's lowest point.

```
function depositEth() external payable nonReentrant whenNotPaused {
```

After the presale starts, users can call `depositEth` to contribute ETH to the presale.

```
function performUpkeep(bytes memory /* performData */) external {
```

Once the presale ends, anyone can call `performUpkeep` to settle the presale.

```
function swapETHForDUTCH(uint256 exactDutchAmount_, address recipient_)
    external
    payable
    whenNotPaused
    nonReentrant
    returns (uint256 ethUsed_)
{
```

During this process, the `swapETHForDUTCH` function in the `Bonding Hook` is invoked to exchange the presale ETH for `DUTCH` tokens without any tax fee.

```
function _createVestingStreams() internal {
```

Afterwards, `_createVestingStreams` is called to create streams, which use `Sablier V2` to distribute `DUTCH` tokens to presale participants.

7 Issues

7.1 [High] Contribution oversubscription will cause `calimReward(...)` call revert due to insufficient funds for late callers

File(s): `AuctionAssist.sol`

Description: `recordPurchase(...)` function allows total contribution shares to exceed 100% when multiple users contribute to a collection. The function caps each individual user's share at 10,000 BPS (100%), but does not normalize the total when the sum of all shares exceeds 10,000 BPS.

```
function recordPurchase(...) external returns (uint256 purchaseId_) {
    //...
    // 7. Calculate and store contributor shares based on actual cost.
    for (uint256 i; i < contributors_.length; ++i) {
        address contributor_ = contributors_[i];
        uint256 contributionAmount_ =
            _contributions[contributor_][collection_].amount;

        if (contributionAmount_ > 0) {
            // Calculate share as (contribution * 10000) / cost.
            // @audit The total of `contributionAmount_` may be greater than `costETH_`.
            uint256 shareBPS_ =
                (contributionAmount_ * 10000) / costETH_;

            // Cap at 10000 BPS (100%) to prevent over-distribution.
            if (shareBPS_ > 10000) {
                shareBPS_ = 10000;
            }

            _purchaseShares[purchaseId_][contributor_] = shareBPS_;
            purchase_.contributors.push(contributor_);
        }
    }
    //...
}
```

If user A contributed 50% of the NFT price and user B contributed 60%, the late `claimReward(...)` caller's transaction will either revert, or pull the tokens that belong to other users from different contributed settlements, causing their claim call to revert.

Impact: Contribution oversubscription will cause 100% fund loss for late `claimReward(...)` callers.

Recommendation(s): Remove contribution oversubscription option for a single purchase or normalize shares proportionally when total exceeds 10,000 BPS.

Status: Fixed

Update from Dutch: Fixed in 74ee906fd0f0588124acef2486a951415685e31d

7.2 [High] Contributions not reset after purchase allows infinite reward claims

File(s): `AuctionAssist.sol`

Description: The `recordPurchase(...)` function calculates contributor shares based on their contribution amounts stored in `_contributions[contributor_][collection_].amount`. However, after recording a purchase and snapshotting the shares, these contribution amounts are never reset.

```
function recordPurchase(...) external returns (uint256 purchaseId_) {
    // ...
    for (uint256 i; i < contributors_.length; ++i) {
        address contributor_ = contributors_[i];
        // @audit contribution amount is read but never reset
        uint256 contributionAmount_ =
            _contributions[contributor_][collection_].amount;

        if (contributionAmount_ > 0) {
            uint256 shareBPS_ = (contributionAmount_ * 10000) / costETH_;
            // ...
            _purchaseShares[purchaseId_][contributor_] = shareBPS_;
            purchase_.contributors.push(contributor_);
        }
    }
    // @audit no reset of _contributions[contributor_][collection_].amount
}
```

Impact: This allows a contributor who made a single contribution to receive reward shares for every subsequent NFT purchase from that collection indefinitely:

a. User contributes 1 ETH to Collection A;
b. NFT 1 is purchased for 2 ETH - User receives 50% share (1 ETH / 2 ETH);
c. NFT 2 is purchased for 2 ETH - User still has 1 ETH recorded, receives another 50% share;
d. This repeats for every future purchase, draining rewards meant for new contributors;

The vulnerability effectively allows infinite DUTCH token extraction from a single contribution, severely diluting rewards for legitimate contributors and draining protocol funds.

Recommendation(s): Reset contribution amounts after recording a purchase

Status: Fixed

Update from Dutch: Fixed in commit b5bacb594faa34b4d77a87588f0b8fbd0b781b6d

7.3 [High] DOS in `_createVestingStreams(...)` due to unbounded loop

File(s): `Presale.sol`

Description: The `_createVestingStreams(...)` function iterates over all contributors in a single transaction to create Sablier vesting streams. Each stream creation costs approximately 175,000-250,000 gas.

```
function _createVestingStreams(uint256 totalTokens) internal {
    uint256 contributorCount = contributors.length;

    // @audit unbounded loop over all contributors
    for (uint256 i = 0; i < contributorCount; i++) {
        address contributor = contributors[i];
        // ...
        // @audit each call costs ~175,000-250,000 gas
        try sablierV2.createWithDurationsLL(params, unlockAmounts, durations)
            returns (uint256 streamId) {
            // ...
        } catch {
            // ...
        }
    }
}
```

With a block gas limit of 30 million, the function can only handle approximately 120-170 contributors before reverting. An attacker can exploit this by creating 200+ small contributions from different addresses during the presale. When the presale ends and `_createVestingStreams(...)` is called, the transaction reverts due to exceeding the block gas limit. This permanently locks all presale funds (both ETH and DUTCH tokens) with no recovery mechanism.

Impact: Lock of funds of all contributors.

Recommendation(s): Implement a batched claiming pattern where each contributor claims their own vesting stream:

Status: Fixed

Update from Dutch: Fixed in: 5f12a82ae4dc706ae4a0891b272b22f69b0cea7f

7.4 [High] Incorrect `SpecifiedDelta` returned

File(s): `DUTCHBondingHook.sol`

Description: In `DUTCHBondingHook`, the `_beforeSwap(...)` function actually executes the user's swap using a custom curve. Therefore, it needs to return a `BeforeSwapDelta` to offset the user's `params_.amountSpecified` input and transfer the AccountDelta generated by the hook to the sender.

```
function _executeBuy(...) internal returns (...) {
    //...
    if (params_.amountSpecified > 0) {
        // Exact input: specify input, calculated output.
        delta_ = toBeforeSwapDelta(
            int128(-params_.amountSpecified),
            int128(int256(outputAmount_))
        );
    } else {
        // Exact output: calculated input, specify output.
        delta_ = toBeforeSwapDelta(
            -int128(int256(inputAmount_)),
            int128(-params_.amountSpecified)
        );
    }
}
function _executeSell(...) internal returns (...) {
    //...
    if (params_.amountSpecified > 0) {
        // Exact input: specify input, calculated output.
        delta_ = toBeforeSwapDelta(
            int128(-params_.amountSpecified),
            int128(int256(outputAmount_))
        );
    } else {
        // Exact output: calculated input, specify output.
        delta_ = toBeforeSwapDelta(
            -int128(int256(inputAmount_)),
            int128(-params_.amountSpecified)
        );
    }
}
```

However, the `_executeBuy(...)` and `_executeSell(...)` functions construct the `BeforeSwapDelta` incorrectly. The high 128 bits of `BeforeSwapDelta` represent `SpecifiedDelta`, which must cancel out the entire `params_.amountSpecified` inside `beforeSwap(...)` so that the AMM swap does not actually occur in Uniswap V4. Therefore, it should always be the exact opposite of `params_.amountSpecified`. In the branches of `_executeBuy(...)` and `_executeSell(...)` where `params_.amountSpecified < 0`, the high 128 bits of `BeforeSwapDelta` are not the opposite of `params_.amountSpecified`. This causes `params_.amountSpecified` to fail to be adjusted to 0, resulting in the AMM swap actually executing. Additionally, the lower 128 bits of `BeforeSwapDelta` represent the `UnspecifiedDelta`, which is used in `afterSwap(...)`. It should always have the opposite sign of the hook's target token `AccountDelta`, so that the hook's debt is correctly transferred to the sender.

Impact: `BeforeSwapDelta` does not return the correct value, causing the swap to fail because it cannot follow the intended execution flow.

Recommendation(s): It is recommended that when constructing `BeforeSwapDelta`, the high 128 bits should always be the opposite of `params_.amountSpecified`, and the low 128 bits should be the opposite of the hook's target token `AccountDelta`.

Status: Fixed

Update from Dutch: Fixed in 287dbcb65f7aa549afe3595493d0c09ca5bac068

Update from CODESPECT: Fixed due to the removal of the hook design.

7.5 [High] Inverted sign convention for `amountSpecified` breaks swap logic

File(s): `DUTCHBondingHook.sol`

Description: The `_executeBuy(...)` and `_executeSell(...)` functions interpret the sign of `params.amountSpecified` to determine swap type. In Uniswap V4, the sign convention is:

- Negative: Exact Input -> User specifies exact amount to pay;
- Positive: Exact Output -> User specifies exact amount to receive;

The hook inverts this convention:

```
function _executeBuy(...) internal returns (...) {
    // ...
    // @audit inverted - positive should be exact output, not exact input
    if (params_.amountSpecified > 0) {
        // Hook treats this as "Exact Input"
        inputAmount_ = uint256(params_.amountSpecified);
        // ...
    } else {
        // Hook treats this as "Exact Output"
        outputAmount_ = uint256(-params_.amountSpecified);
        // ...
    }
}
```

The V4 source code confirms the correct convention in `lib/v4-core/src/libraries/Hooks.sol`:

Impact: This causes the specified and unspecified amounts to be swapped together in the `afterSwap` hook in v4, eventually causing the transaction to revert.

Recommendation(s): Invert the condition to match V4's sign convention:

```
if (params_.amountSpecified < 0) {
    // Exact Input - user specifies input amount (negative in V4)
    inputAmount_ = uint256(-params_.amountSpecified);
    // ... exact input logic
} else {
    // Exact Output - user specifies output amount (positive in V4)
    outputAmount_ = uint256(params_.amountSpecified);
    // ... exact output logic
}
```

Same fix can be applied ot `_executeSell(...)`

Status: Fixed

Update from Dutch: Fixed in commit c79da4114ffd2b47c5fd313bfd38ec1ca8540044

Update from CODESPECT: Fixed due to the removal of the hook design.

7.6 [High] Tax fee revenue may be lost due to Uniswap V4 trading

File(s): DUTCHBondingHook.sol

Description: By design, mainstream dex are expected to be blacklisted for DUTCH Tokens to restrict off-market trading, ensuring that all trades go through the DUTCHBondingHook and incur a tax fee. The DUTCHBondingHook overrides _beforeSwap(...) so that when a user trades via Uniswap V4 swap, the calculation does not go through the AMM but is performed using a custom curve.

```
function _executeBuy(...) internal returns (...) {
    //...
    // Take WETH from pool (user already deposited it).
    poolManager.take(_wethCurrency, address(this), inputAmount_);

    // Distribute tax.
    (uint256 vaultAmount_, uint256 opsAmount_) =
        _calculateBuyTax(taxAmount_);
    IERC20(Currency.unwrap(_wethCurrency)).safeTransfer(
        _dutchVault,
        vaultAmount_
    );
    IERC20(Currency.unwrap(_wethCurrency)).safeTransfer(
        _opsWallet,
        opsAmount_
    );

    // Mint DUTCH to hook.
    dutchToken.mint(address(this), outputAmount_);
    //...
}
```

During the purchase process, DUTCHBondingHook calls take(...) to withdraw WETH from the pool, mints DUTCH to send to the pool, and calls settle(...) to update the account. The Hook accountDelta are transferred to the sender in afterSwap. Afterwards, the sender must repay the WETH to the Hook and withdraw DUTCH from the pool. The above process prevents the Uniswap V4 PoolManager from being blacklisted for DUTCH tokens, because this contract must handle sending tokens to the buyer during the purchase process.

Impact: This allows off-market trading to occur on Uniswap V4. Users can create DUTCH related pools and trade without using the DUTCHBondingHook, causing the protocol to lose tax fee revenue.

Recommendation(s): It is recommended to send DUTCH directly to the user during the purchase process, instead of involving the PoolManager in the DUTCH token transfer. This way, the PoolManager can be blacklisted to prevent off-market trading on Uniswap V4.

Status: Fixed

Update from Dutch: Fixed in 9da861422e64236acfe61feb3efcc0ccbbe5f49a

Update from CODESPECT: Fixed. Since the Hook design was removed, Uniswap V4 can be added to the blacklist.

7.7 [High] settleWithETH(...) function call will revert because of ETH refunds

File(s): DUTCHBondingHook.sol, DutchAuctionMarketplace.sol

Description: settleWithETH(...) forwards all msg.value to swapETHForDUTCH(...) function, which refunds unused ETH to DutchAuctionMarketplace after the swap. This will cause NFT purchases with ETH to revert because DutchAuctionMarketplace contract doesn't implement receive() or fallback() functions.

Impact: settleWithETH(...) function DoS.

Recommendation(s): Implement receive() function in DutchAuctionMarketplace contract.

Status: Fixed

Update from Dutch: Fixed in: ac0dda10392af280b44d249e70230e45660fc793

7.8 [Medium] Bonding-curve taxes and burn revenue cannot be distributed within the collection

File(s): DUTCHBondingHook.sol

Description: Bonding-curve taxes and burn revenue are sent to the `DutchVault`. These revenues are distributed according to the basis-point weights configured by the owner, increasing the available funds for NFT purchases in each collection. This process is carried out in the `receive()` function.

```
function withdrawExcessReserve(uint256 amount_) external onlyOwner {
    //...
    IERC20(Currency.unwrap(_wethCurrency)).safeTransfer(
        _dutchVault,
        amount_
    );
    emit ExcessReserveWithdrawn(amount_, _dutchVault);
}

function _executeBuy(...) internal returns (...) {
    //...
    // Distribute tax.
    (uint256 vaultAmount_, uint256 opsAmount_) =
        _calculateBuyTax(taxAmount_);
    IERC20(Currency.unwrap(_wethCurrency)).safeTransfer(
        _dutchVault,
        vaultAmount_
    );
   //...
}

function _executeSell(...) internal returns (...) {
    //...
    // Distribute tax.
    (uint256 vaultAmount_, uint256 opsAmount_) =
        _calculateBuyTax(taxAmount_);
    IERC20(Currency.unwrap(_wethCurrency)).safeTransfer(
        _dutchVault,
        vaultAmount_
    );
   //...
}
```

However, when sending bonding-curve taxes and burn revenue, WETH is transferred directly instead of being unwrapped into ETH.

Impact: Bonding-curve taxes and burn revenue are sent as WETH to the `DutchVault`, and there is no function in `DutchVault` to unwrap WETH. This causes the revenue distribution to not actually occur, leaving the WETH stuck.

Recommendation(s): It is recommended to unwrap the revenue into ETH before sending it to the `DutchVault`.

Status: Fixed

Update from Dutch: Fixed in commit 734d928fa3344927a8ac14a402778486cdc8e6cf

7.9 [Medium] Inventory costETH records max price instead of actual cost causing cascading errors

File(s): DutchVault.sol, AuctionAssist.sol

Description: When _buyNFTInternal(...) purchases an NFT, it records costETH as the maximum specified value_ parameter and passes this same inflated value to AuctionAssist. If the marketplace refunds excess ETH, the recorded cost does not reflect the actual price paid.

```
function _buyNFTInternal(...) internal {
    // ...
    // @audit Records max price before purchase executes
    _inventory[collection_][tokenId_] = InventoryRecord({
        // ...
        costETH: value_,  // Max price, not actual
        // ...
    });

    // Marketplace call - may refund excess
    (bool success,) = marketplace_.call{value: value_}(marketplaceCalldata_);
    // Refund goes to receive() but costETH is never updated

    // @audit Passes same inflated value to AuctionAssist
    if (_auctionAssist != address(0)) {
        uint256 purchaseId_ = IAuctionAssist(_auctionAssist).recordPurchase(
            collection_, tokenId_, value_  // Inflated cost
        );
    }
}
```

Impact: The inflated costETH propagates through multiple calculations:

a. **AuctionAssist Contributor Shares** (recordPurchase):;

```
uint256 shareBPS_ = (contributionAmount_ * 10000) / costETH_;
// @audit Inflated denominator reduces contributor shares
```

b. **Listing Prices** (_listNFTOnMarketplace):;

```
uint256 maxPrice_ = (costETH_ * _upperAuctionMultiplierBps) / _BPS_DENOMINATOR;
uint256 minPrice_ = (costETH_ * _lowerAuctionMultiplierBps) / _BPS_DENOMINATOR;
// @audit NFTs listed higher than necessary, reducing sales
```

c. **Profit/Loss Tracking** (_processSettlement):;

```
bool isProfit_ = salePriceETH_ >= costETH_;
profitOrLoss_ = isProfit_ ? salePriceETH_ - costETH_ : costETH_ - salePriceETH_;
// @audit Profits understated, losses overstated
```

d. **Contributor Share in Settlement** (_splitProceeds):;

```
uint256 contributorShareBPS_ = (totalContributions_ * 10000) / costETH_;
// @audit Contributors receive diluted reward shares
```

Recommendation(s): Track actual ETH spent by measuring the balance before/after the marketplace call.

Status: Fixed

Update from Dutch: Fixed in ffa02072c6b0c681673c8bdc44de8e810bcbeb20

7.10 [Medium] Marketplace refunds misattributed across collections causing contributor reward loss

File(s): DutchVault.sol

Description: When `_buyNFTInternal(...)` purchases an NFT, it deducts the full specified `value_` from the target collection's balance before executing the marketplace call. If the actual purchase price is less than `value_`, the marketplace refunds the excess ETH to the vault.

```
function _buyNFTInternal(...) internal {
    // ...
    // Full value deducted from specific collection
    _collectionBalances[collection_] -= value_;

    // Marketplace call - may refund excess
    (bool success,) = marketplace_.call{value: value_}(marketplaceCalldata_);
    // ...
}
```

The refund arrives at the vault's `receive()` function, which distributes incoming ETH proportionally across ALL collections based on their allocation percentages:

```
receive() external payable {
    // ...
    for (uint256 i; i < _collectionList.length; ++i) {
        address collection_ = _collectionList[i];
        uint16 allocation_ = _allocations[collection_];
        uint256 share_ = (msg.value * allocation_) / _BPS_DENOMINATOR;
        _collectionBalances[collection_] += share_;  // All collections receive share
    }
    // ...
}
```

Impact: Contributors who funded Collection A through AuctionAssist receive reduced rewards because their collection's balance is understated. Meanwhile, Collection B contributors receive unearned gains. Over multiple purchases with refunds, this drift accumulates, causing systematic unfairness to active collection contributors.

Recommendation(s): Track the actual ETH spent and credit any refund back to the originating collection.

Status: Acknowledged

Update from Dutch: Issue fixed: a970cba006ae1bbc0145015bd7b8495d47d359e5

Update from CODESPECT: This fix is incorrect and may lead to double accounting. For example:

a. The vault sends 1 ETH to `marketplace_` (`value = 1 ETH`);
b. Purchasing the NFT costs 0.95 ETH, with a 0.05 ETH refund;
c. The refund triggers the `receive` function, which distributes it across collections;
d. After the call completes, `collectionBalances[collection_]` records an additional 0.05 ETH refund again;

Update from Dutch: I think for simplicity, we are fine with redistributing the refund across all collections. The refund will be small and in many cases, there will be no refund at all. Fixed in: PR-126

7.11 [Medium] Stuck NFTs block rebalancing and write-off workaround causes contributor reward loss

File(s): DutchVault.sol

Description: The `_rebalanceCollections(...)` function prevents removing any collection that still holds NFTs. If a collection has even one NFT in inventory, it cannot be removed from the allocation list.

```
function _rebalanceCollections(...) internal {
    // Mark new collections temporarily using max value.
    for (uint256 i; i < collections_.length; ++i) {
        _allocations[collections_[i]] = type(uint16).max;
    }

    for (uint256 i; i < _collectionList.length; ++i) {
        address oldCollection_ = _collectionList[i];

        // @audit If ANY collection being removed has NFTs, entire rebalance reverts
        if (
            _allocations[oldCollection_] != type(uint16).max
                && _itemsHeld[oldCollection_] > 0
        ) {
            revert CollectionHasNFTs();
        }
        // ...
    }
}
```

If an NFT becomes unsellable (marketplace delists it, collection rugpulls, floor price crashes, legal issues), that collection can never be removed from allocations. This blocks ALL rebalancing operations for the entire vault.

Workaround causes contributor loss: The owner can call `recordSettlement(collection, tokenId, 0, 0)` to manually "settle" a stuck NFT at zero price. However, this causes real fund loss:

a. **Contributor reward loss**: If AuctionAssist contributors funded the NFT purchase with ETH, settling at 0 means `salePriceDUTCH_ = 0`, so contributors receive 0 DUTCH rewards despite funding the purchase;
b. **Vault retains asset**: The vault still physically holds the NFT (which may have value), but contributors get nothing;
c. **Inflated losses**: Full `costETH` is recorded as realized loss, overstating the protocol's trading losses;
d. **No recovery path**: If the NFT is later sold through other means, there's no mechanism to credit contributors retroactively;

Impact: This can prevent rebalancing from being possible. And the workaround causes the vault to incur total loss of the asset.

Recommendation(s): Add a dedicated `writeOffNFT(...)` function that cleanly handles stuck inventory.

Status: Fixed

Update from Dutch: Fixed in commit f88609069273aeb2cbb8df62dcd950018da95d21

7.12 [Medium] The `contribute(...)` function may be DoS

File(s): `AuctionAssist.sol`

Description: In `AuctionAssist`, each collection can have a maximum of 100 contributors to prevent excessive loops from causing gas exhaustion.

```
function contribute(address collection_) external payable nonReentrant whenNotPaused{
    //...
    if (!_hasContributed[collection_][msg.sender]) {
        // Check max contributors limit.
        if (_collectionContributors[collection_].length >= MAX_CONTRIBUTORS)
        {
            revert MaxContributorsExceeded();
        }
    //...
}
```

However, a malicious actor can switch accounts and contribute 1 wei 100 times to reach the contributor limit, preventing funds from being raised.

Impact: The fund-raising functionality of `AuctionAssist` may be vulnerable to a DoS attack.

Recommendation(s): It is recommended to refactor the code logic and use alternative methods to avoid excessive gas consumption.

Status: Fixed

Update from Dutch: Fixed by commit 8e583e00553d9178e8204bacda50fdfcaffae9f4

Update from CODESPECT: We believe this issue has only been **partially fixed**:

Although setting a **minimum contribution amount** mitigates **dust attacks** to some extent, **AuctionAssist may still become unusable in the long run**. The reasons are:

- Each collection allows a maximum of **100 contributors**
- The **contributors array has no cleanup mechanism** and can only continue to grow In this situation, even if some contributors' balances are reduced to zero, they still occupy slots in the array. Over time, this may prevent new valid contributors from being added, ultimately blocking the contract's functionality. Additionally, an attacker can still increase the array length by repeatedly calling `contribute` and then immediately `withdrawContribution`.

Recommendations:

- Add an **cleanup mechanism** to remove contributors from the array when their balances are zeroed
- Additionally, provide an **admin-controlled queue cleanup mechanism** to manually remove malicious contributor records when necessary.

Update from Dutch: Fixed in PR-124

Update from CODESPECT: The above fix initially establishes a clearing mechanism, and it is also recommended to add an admin clearing mechanism for withdrawing small `_contributions`. Additionally, when processing refunds, if sending ETH fails in admin clear, the funds that were not successfully sent are stored in a mapping to allow later claims, preventing DoS.

Update from Dutch: Good call, pushed into the same PR e04b86410fcf5c6d066d4908c67b968071864f08

7.13 [Medium] The `swapETHForDUTCH` function does not charge royalty fees

File(s): `DUTCHBondingHook.sol`

Description: Purchasing DUTCH through Uniswap V4 requires paying a 10% tax fee. The `swapExactETHForDUTCH(...)` function, used in `DutchAuctionMarketplace` to swap ETH for DUTCH when buying NFTs, does not incur the tax fee.

```
function swapETHForDUTCH(...)
    external
    payable
    whenNotPaused
    nonReentrant
    returns (uint256 ethUsed_)
{
    //...
    // 1. Calculate ETH needed (no tax for marketplace).
    ethUsed_ = _calculateMintCost(exactDutchAmount_);
    //...
}
```

However, in `DutchAuctionMarketplace`, listing and settlement lack access control. A user can list their own NFT, set `minPrice` to 0 to avoid paying the listing fee, and then call `settleWithETH(...)` to buy their own listed NFT. The ETH they input will be swapped for DUTCH with 0 tax fee, and they only need to pay a 2.5% marketplace fee. This allows them to acquire DUTCH while bypassing most of the tax fee.

Impact: `swapExactETHForDUTCH(...)` could be exploited to acquire DUTCH at a lower fee.

Recommendation(s): It is recommended to apply the tax fee in `swapETHForDUTCH(...)` for listings that are not from `DutchVault`.

Status: Fixed

Update from Dutch: Fixed in commit 2dfad0d5e39282955bd2a1098d5d9d249be003ff

Update from Dutch: The preview fix is reverted and a new is implemented in PR-99 Previously, we added tax to all settleWithETH purchases. That is undesired behaviour. In the new PR, we keep no tax, but only protocol listings can be bought with ETH. This prevents this attack vector. Also, in this PR we add a view method on the hook to get the current no tax ETH needed for the purchase.

Update from Dutch: Fixed in ee8512b51c68567c69237d215e56754d2cd2ca96

7.14 [Medium] User's contribution can be rebalanced to other collections

File(s): `DutchVault.sol`

Description: `setCollectionAllocations(...)` function has an option to rebalance the funds through collections but it doesn't check if there are any user contributions to the collection, nor it modifies user's contribution data in `AuctionAssist`. Users' contributions are bound to the collection address they have contributed to, and if admin decides to rebalance that allocation to other collections user contribution mapping will still point to the old inactive collection. This will cause users not being able to claim the rewards for their contribution, and because there is no option to withdraw funds from inactive collections, users will lose their contributions.

Impact: Users can lose 100% of their contributions.

Recommendation(s): Consider excluding user contributions from rebalancing calculations and adding an option for users to withdraw their funds from inactive collections.

Status: Fixed

Update from Dutch: Fixed in ceac1ccf5ba85cbac5e71f160a58959c3aed6207

Update from Dutch: Regression bug fixed here: https://github.com/dutch-protocol/Protocol-Contracts/issues/114

7.15 [Low] Bad ownership check will prevent `DutchVault` from buying NFTs that were purchased in the past

File(s): `DutchVault.sol`

Description: `_buyNFTInternal(...)` function performs following NFT ownership check:

```
InventoryRecord storage existing_ = _inventory[collection_][tokenId_];
if (existing_.costETH > 0) {
    revert NFTAlreadyOwned();
}
```

This will cause `DutchVault` being unable to buy back and sell NFTs that were purchased in the past because of the previously created `_inventory[collection_][tokenId_]` record. The check also creates attack surface by transfering NFT directly to the contract or making the `DutchVault` to purchase the NFT for free.

Impact: Vault will be unable to buy the same NFT again.

Recommendation(s): The check should look like this:

```
if (IERC721(collection_).ownerOf(tokenId_) == address(this)) {
    revert NFTAlreadyOwned();
}
```

Status: Fixed

Update from Dutch: It is now possible to buy the same NFT multiple times 2346c1e0379cba853a153f9914a0011ea07b84af

7.16 [Low] Failed vesting stream creation causes permanent loss of contributor funds

File(s): `Presale.sol`

Description: The `_createVestingStreams(...)` function iterates through all contributors to create Sablier vesting streams for their token allocations. If stream creation fails for any contributor, the function catches the error, emits an event, and continues to the next contributor. However, there is no fallback mechanism to recover the funds of the failed contributor.

```
function _createVestingStreams() internal {
    // ...
    for (uint256 i = 0; i < contributorCount; i++) {
        address contributor = contributors[i];
        uint256 userTokenAllocation = (totalTokens * userContribution) / totalFundedAmount;

        // ...

        try sablierV2.createWithDurationsLL(params, unlockAmounts, durations)
            returns (uint256 streamId) {
            vestingStreamIds[contributor] = streamId;
            streamsCreated++;
            emit TokensClaimed(contributor, userTokenAllocation);
        } catch {
            // @audit Tokens stuck - no recovery mechanism
            emit StreamCreationFailed(contributor, userTokenAllocation);
            continue;

            // ! no fallback method to return the funds to the user
        }
    }
}
```

Impact: Contributors who experience failed stream creation permanently lose both their ETH contribution and their token allocation. The tokens remain stuck in the Presale contract with no recovery path.

Recommendation(s): Implement a fallback claiming mechanism for failed streams.

Status: Fixed

Commit b6ec15a6aa038829722c1d34079aba58cdcaca6b

7.17 [Low] Fake NFT contract attack due to missing ownership verification

File(s): DutchAuctionMarketplace.sol

Description: The `_settleInternal(...)` function transfers an NFT from the seller to the buyer when an auction is settled. After calling `transferFrom(...)` on the NFT contract, the function burns the buyer's DUTCH tokens as payment.

```
function _settleInternal(...) internal {
    // ...
    // @audit no ownership verification after transfer
    IERC721(listing_.nftContract).transferFrom(
        listing_.seller, buyer_, listing_.tokenId
    );

    // Buyer's DUTCH tokens are burned regardless of transfer success
    _DUTCH.burn(buyer_, listing_.currentPrice);
    // ...
}
```

However, there is no verification that the NFT transfer actually succeeded.

Impact: A malicious seller can deploy a fake ERC721 contract with a `transferFrom(...)` function that always returns success but never actually transfers any tokens. When a victim bids on this fake NFT listing:

a. The attacker creates a listing with their fake NFT contract;
b. The victim places a bid on what appears to be a legitimate NFT;
c. On settlement, the fake `transferFrom(...)` succeeds but transfers nothing;
d. The victim's DUTCH tokens are burned as payment;
e. The attacker receives the payment while the victim receives nothing;

This results in a complete loss of the buyer's DUTCH tokens with no recourse.

Recommendation(s): Add ownership verification after the transfer:

```
IERC721(listing_.nftContract).transferFrom(
    listing_.seller, buyer_, listing_.tokenId
);

require(
    IERC721(listing_.nftContract).ownerOf(listing_.tokenId) == buyer_,
    "NFT transfer failed"
);
```

Status: Fixed

Update from Dutch: Fixed in: 7b9105de3aaeb5808a76caeb75ef0016bc9777b1

7.18 [Low] Inappropriate slippage control is present in `settleWithETH(...)`

File(s): `DutchAuctionMarketplace.sol`

Description: `settleWithETH(...)` allows an NFT buyer to send ETH to purchase an NFT. The ETH is converted into DUTCH via the `DUTCHBondingHook` to settle for users in `AuctionAssist`.

```
function settleWithETH(uint256 listingId_, uint256 maxPriceETH_) external payable whenNotPaused nonReentrant
{
    //...
    // 5. Get current NFT price, convert to DUTCH, and check slippage.
    CurrentPrice memory currentPrice_ = _getCurrentPrice(listingId_);
    if (currentPrice_.priceETH > maxPriceETH_) revert SlippageExceeded();

    // 6. Mark as settled (before external calls).
    listing_.settled = true;

    // 7. Swap ETH for exact DUTCH amount via bonding hook.
    // Hook will refund excess ETH back to this contract.
    uint256 ethUsed_ = _bondingHook.swapETHForDUTCH{value: msg.value}(
        currentPrice_.priceDUTCH,
        address(this)
    );
    //...
}
```

However, during the slippage check, the NFT's ETH price is used directly for comparison. The slippage from the DUTCH price in the transaction is not accounted for and cannot be controlled via parameters.

Impact: The actual ETH paid by the user may exceed the slippage parameter `maxPriceETH_`.

Recommendation(s): It is recommended to perform the slippage check by comparing `ethUsed_` with `maxPriceETH_`.

Status: Fixed

Update from Dutch: Fixed in: 1b78ffeecea23ba20aaec534a580b35366dc6bda

7.19 [Low] It's possible to create duplicate listings in `DutchAuctionMarketplace`

File(s): `DutchAuctionMarketplace.sol`

Description: `_createListing(...)` function doesn't check if the `tokenId_` of the `nftContract_` is currently listed or not and lets users to create duplicate listings, which will remain in the marketplace even after the NFT will be sold.

Impact: This will cause gas griefing on users that will try to purchase the NFT using duplicate listings.

Recommendation(s): Check if the NFT is already listed in the `DutchAuctionMarketplace`.

Status: Fixed

Update from Dutch: Fixed in 28b610ddfda26c7d47f0915e79de67ed57b2bb53

7.20 [Low] The `presale(...)` function may be DoS

File(s): `Presale.sol`

Description: The presale is expected to be the first DUTCH purchase transaction to prevent users from frontrunning and buying DUTCH at the lowest price for profit. Therefore, when presale calculating the expected DUTCH output, a supply of 0 is used as the starting point by default. To prevent DUTCH purchases before the presale, the project team needs to pause the system. However, during the process of unpausing the system and calling `performUpkeep(...)` to settle the presale, if a malicious actor frontruns by calling `settleWithETH` or making a purchase, causing the DUTCH supply to be nonzero. Then `performUpkeep(...)` could fail due to slippage control.

Impact: `performUpkeep(...)` may fail due to frontrunning, causing the presale funds to be locked.

Recommendation(s): It is recommended to add an `isPresaleEnd` field in `DUTCHBondingHook` and prohibit calls to `_beforeSwap` and `swapETHForDUTCH` while this field is `false`. The field would be set to `true` after `swapExactETHForDUTCH` is called.

Status: Fixed

Update from Dutch: Fixed in commit 03c6753d1d7677f2d16d4aaf0b33e7238256d4d1

7.21 [Low] Unused `vaultFee`

File(s): `DutchAuctionMarketplace.sol`

Description: In `DutchAuctionMarketplace`, selling an NFT incurs a `_sellerFeeOnSettled`. In the `_settleInternal(...)` function, a portion of `_dutchToken` is sent to the `dutchVault` as `vaultFee_`.

```
function _settleInternal(...) internal {
    //...
        if (vaultFee_ != 0) {
            IERC20(address(_dutchToken)).safeTransfer( //NOTE
                _dutchVaultAddress, vaultFee_
            );
        }
    //...
}
```

However, in `dutchVault`, this portion of `_dutchToken` fees is not utilized.

Impact: The `_dutchToken` fees that are sent will be stuck.

Recommendation(s): It is recommended to merge `vaultFee_` into `burnFee` and burn the corresponding `_dutchToken`.

Status: Acknowledged

Update from Dutch: We acknowledge this issue. We are aware, that dutch vault cannot currently handle incoming DUTCH token, but we keep it in the fee split for potential future replacement of DutchVault with a new implementation, that would need the DUTCH token to be send to it.

7.22 [Low] `_buyNFTInternal(...)` function uses unchecked calldata for NFT purchases

File(s): `DutchVault.sol`

Description: `_buyNFTInternal(...)` function receives arbitrary calldata and uses it to purchase NFTs from the marketplaces:

```
(bool success,) =
    marketplace_.call{value: value_}(marketplaceCalldata_);
if (!success) revert MarketplaceCallFailed();
```

The function doesn't validate the arbitrary calldata, which means that any function with any parameters can be called on approved marketplaces. Attack scenario:

a. Attacker waits for approved collection's `getMaxPriceForCollection(...)` to be high enough or an NFT listed below the floor price to pass the `value_ > this.getMaxPriceForCollection(collection_)` check;
b. Calls marketplace function that batch buys NFTs;
c. Besides purchasing the NFT for the vault from approved collection, attacker also buys unvalued NFT that he listed on the marketplace;

Impact: Attacker can profit by making the vault buy unvalued NFTs

Recommendation(s): Validate `marketplaceCalldata_` parameter.

Status: Fixed

Update from Dutch: Fixed in commit 5dd04c73343a03b823f6b1037dc52d60696bb620

Update from CODESPECT: Since the unverified calldata may still potentially expand the attack surface, we still recommend using a `(market address => (function signature => bool))` mapping to further restrict callable functions and reduce attack risks.

Update from Dutch: Fixed in PR-125 We changed the marketplace address allowlist to [address, selector] allowlist.

7.23 [Low] `record.listed` status is not updated after the NFT is sold

File(s): `DutchVault.sol`

Description: After the vault's listed NFT is purchased contract doesn't update the `_inventory[collection_][tokenId_].listed` status to false, although it's not listed anymore. This will cause `DutchVault` being unable to list previously sold NFTs again because of following check in the `listNFTOnMarketplace(...)`:

```
if (record_.listed) {
    revert NFTAlreadyListed();
}
```

Impact: `DutchVault` can't list previously sold NFTs again.

Recommendation(s): Set `_inventory[collection_][tokenId_].listed` to false after NFT got purchased

Status: Fixed

Update from Dutch: Fixed in 26db416a8d185792721a1da1f9622eb8252f7838

7.24 [Low] `withdrawCollectionBalance(...)` can withdraw user contributions

File(s): `DutchVault.sol`

Description: `withdrawCollectionBalance(...)` function doesn't check if the collection has any user contributions before withdrawing the balance. This can cause owner mistakenly withdrawing user contributions.

Impact: Users can lose 100% of their contributions.

Recommendation(s): Check if there are any user contributions for the collection and let the owner to only withdraw protocol funded assets.

Status: Fixed

Update from Dutch: Fixed here PR-89

7.25 [Info] Improper collection configuration checks

File(s): `AuctionAssist.sol`

Description: In the `contribute(...)` function, the collection configuration is checked by verifying whether `basePrice_` is zero.

```
function contribute(...) external payable nonReentrant whenNotPaused{
    //...
    // 2. Get base price (target price) from DutchVault.
    uint256 basePrice_ = IDutchVault(_dutchVault).getBasePrice(collection_);

    // 3. Validate collection is configured.
    if (basePrice_ == 0) {
        revert CollectionNotConfigured();
    }
   //...
}
```

However, when a collection is removed, `basePrice_` may not be cleared.

Impact: This may cause users to mistakenly deposit into an inactive collection.

Recommendation(s): It is recommended to check that `_allocations[collection_]` is not equal to zero.

Status: Fixed

Update from Dutch: Fixed in e38f1cd269761bb4b5901c321cedb83b2eab515e

7.26 [Info] Open TODOs

File(s): `DutchVault.sol`

Description: There are several TODO sections in the code and it's recommended to complete these TODOs before deployment. In particular, the TODO in `_splitProceeds(...)` function, which states that contribution calculation doesn't exclude new contributions that occurred after the purchase. This will cause `recordAndPullRewards(...)` function being called with inflated `contributorAmount_` value.

Impact: Recorded contributors will receive more share from the purchase than they should.

Recommendation(s): Exclude new contributions from the calculation.

Status: Fixed

Update from Dutch: Fixed in 67b3b69cef288936425f47313485286ffb4a87f0

7.27 [Info] The price growth has no upper limit

File(s): DutchVault.sol

Description: If `_buyIncrements[collection_]` is configured, the price the protocol can pay for an NFT will increase over time until an NFT in the collection is purchased, at which point it resets.

```
function getMaxPriceForCollection(address collection_)
    external
    view
    returns (uint256 maxPrice_)
{
    //...
    uint256 timeBuffer_ =
        (blocksSinceLastBuy_ + 1) * _buyIncrements[collection_];
    return basePrice_ + timeBuffer_;
}
```

If NFTs are never purchased, the price will continue to rise without any upper limit.

Impact: If a collection continuously has no NFT purchases due to no NFTs being listed, the protocol's bid could become far higher than the funds raised.

Recommendation(s): It is recommended to set a limit on price growth.

Status: Acknowledged

Update from Dutch: We acknowledge this finding, but consider the current implementation **intentional and safe by design**.

The `getMaxPriceForCollection` function represents the protocol's **maximum willingness to pay** (a market signal), not a guarantee of payment capability. While this value can grow unbounded over time, actual purchases in `_buyNFTInternal` are protected by a critical balance check (`if (_collectionBalances[collection_] < value_) revert InsufficientBalanceForPurchase();`) that ensures the protocol can never spend more than it has, regardless of how high `getMaxPriceForCollection` grows. The unbounded growth in willingness to pay is intentional, and we believe the market should determine pricing without artificial protocol-imposed limits. The real constraint (available treasury funds) is what matters, not an artificial cap on willingness to pay. If `getMaxPriceForCollection` grows beyond available funds, purchases will simply fail until more funds are deposited or the price resets via a purchase, and the protocol owner can always adjust `_buyIncrements[collection_]` or reset `_lastBuyBlock[collection_]` if needed.

7.28 [Info] `_lastBuyBlock` is not fully updated

File(s): DutchVault.sol

Description: `_lastBuyBlock` stores the last purchase block for each collection. The protocol allows the maximum price for purchasing NFTs from that collection to increase over the elapsed blocks. When calling the `setCollectionAllocations(...)` function to update collections, only collections with `_lastBuyBlock` equal to 0 will have their `_lastBuyBlock` updated.

```
function setCollectionAllocations(...) external onlyOwner whenNotPaused {
    //...
    for (uint256 i; i < collections_.length; ++i) {
        _allocations[collections_[i]] = allocationsBPS_[i];
        _collectionList.push(collections_[i]);

        // Initialize lastBuyBlock if not set (prevents huge first-buy cap).
        if (_lastBuyBlock[collections_[i]] == 0) {
            _lastBuyBlock[collections_[i]] = block.number;
        }

        emit AllocationUpdated(collections_[i], allocationsBPS_[i]);
    }
}
```

However, if a collection was previously removed and then added again, its `_lastBuyBlock` will not be 0, so `_lastBuyBlock` will not be updated when it is re-added.

Impact: If too much time has passed since `_lastBuyBlock`, the payable price for the collection could become excessively high.

Recommendation(s): It is recommended to update `_lastBuyBlock` for each collection when calling `setCollectionAllocations(...)`.

Status: Fixed

Update from Dutch: Fixed in: f6c7c3e0a42e80ff2cb0d147854a96113a6c276b

8 Evaluation of Provided Documentation

The **DUTCH** documentation was provided in the forms of a README file and NatSpec comments:

- **README:** The provided README offered a comprehensive overview of the protocol's functionality, including a high-level description of the contracts, the main components and flows, as well as detailed installation and testing instructions.
- **NatSpec:** The in-code NatSpec comments were clear, detailed, and highly effective in explaining specific flows, design intentions, and conditional branches. Each core functionality was well documented, allowing for an efficient understanding of the system's behavior and expected invariants.

Overall, the documentation was adequate and fully sufficient for the scope of this audit. Moreover, the DUTCH team remained consistently available and responsive, promptly addressing all questions and clarifications raised by **CODESPECT**, which significantly streamlined the audit process.

9 Test Suite Evaluation

9.1 Compilation Output

```
> forge build
[] Compiling...
[] Compiling 176 files with Solc 0.8.28
[] Solc 0.8.28 finished in 28.13s
Compiler run successful with warnings:
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
   --> src/DutchVault.sol:841:55:
    |
841 |     function receiveContribution(address collection_, address contributor_)
    |                                                       ^^^^^^^^^^^^^^^^^^^
```

9.2 Tests Output

```
> forge test
[] Compiling...
No files changed, compilation skipped

Ran 2 tests for test/DeploySepolia.t.sol:DeploySepoliaTest
[PASS] testDeployment_revertsOnAnvil() (gas: 21065)
[PASS] testDeployment_revertsOnMainnet() (gas: 21153)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.06ms (167.98µs CPU time)

Ran 10 tests for test/Integration.t.sol:IntegrationTest
[FAIL: Rewards funded] test_journey_auctionAssistContribution() (gas: 1758076)
[PASS] test_journey_buyAndListAtomically() (gas: 910097)
[PASS] test_journey_cancelAndReList() (gas: 1251022)
[PASS] test_journey_completeLifecycleFromDepositToBurn() (gas: 1303324)
[PASS] test_journey_dustDistributionFairness() (gas: 171283)
[PASS] test_journey_moneyFlowInvariants() (gas: 569674)
[PASS] test_journey_multiCollectionParallelOperations() (gas: 1666207)
[FAIL: PurchaseNotFunded()] test_journey_multipleContributors() (gas: 1936531)
[PASS] test_journey_pausingFunctionality() (gas: 963196)
[PASS] test_journey_profitAndLossAccounting() (gas: 1738119)
Suite result: FAILED. 8 passed; 2 failed; 0 skipped; finished in 26.34ms (21.46ms CPU time)

Ran 105 tests for test/DutchAuctionMarketplace.t.sol:DutchAuctionMarketplaceTest
[PASS] testBatchSettle_revertsGivenAnyListingFails() (gas: 1533779)
[PASS] testBatchSettle_revertsGivenBatchSizeTooLarge() (gas: 40584)
[PASS] testBatchSettle_worksGivenMultipleListings() (gas: 1511296)
[PASS] testCancelListing_revertsGivenCallerNotSellerOrOwner() (gas: 460624)
[PASS] testCancelListing_revertsGivenInvalidListingId() (gas: 20349)
[PASS] testCancelListing_worksGivenNFTOwnerNotSeller() (gas: 494761)
[PASS] testCancelListing_worksGivenSellerIsOwner() (gas: 479370)
[PASS] testCreateListingWithBurn_revertsGivenFeeIsSent() (gas: 129621)
[PASS] testCreateListingWithBurn_worksGivenAnyone() (gas: 371200)
[PASS] testCreateListingWithBurn_worksGivenOwner() (gas: 367805)
[PASS] testCreateListing_calculatesCorrectFeeForHighMinPrice() (gas: 454271)
[PASS] testCreateListing_revertsGivenCollectionNotWhitelisted() (gas: 154670)
[PASS] testCreateListing_revertsGivenIncorrectETHAmount() (gas: 140439)
[PASS] testCreateListing_revertsGivenMarketplaceIsPaused() (gas: 147473)
[PASS] testCreateListing_revertsGivenMaxPriceLessThanMinPrice() (gas: 27469)
[PASS] testCreateListing_revertsGivenNotApproved() (gas: 108350)
[PASS] testCreateListing_revertsGivenNotNFTOwner() (gas: 98238)
[PASS] testCreateListing_revertsGivenZeroMaxPrice() (gas: 28008)
[PASS] testCreateListing_revertsGivenZeroMinPrice() (gas: 28743)
[PASS] testCreateListing_revertsGivenZeroNFTContract() (gas: 25133)
[PASS] testCreateListing_setsProceedsRecipientToSeller() (gas: 457012)
[PASS] testCreateListing_worksGivenCollectionIsWhitelisted() (gas: 502657)
[PASS] testCreateListing_worksGivenValidNFTOwnershipAndApproval() (gas: 480167)
[PASS] testGetCurrentPrice_revertsGivenCancelledListing() (gas: 463412)
[PASS] testGetCurrentPrice_revertsGivenInvalidListingId() (gas: 16437)
[PASS] testGetCurrentPrice_revertsGivenSettledListing() (gas: 730877)
[PASS] testGetCurrentPrice_worksGivenAuctionEnd() (gas: 465357)
[PASS] testGetCurrentPrice_worksGivenAuctionStart() (gas: 464007)
```

```
[PASS] testGetCurrentPrice_worksGivenDuringAuction() (gas: 467596)
[PASS] testGetCurrentPrice_worksGivenDuringAuctionToZero() (gas: 633779)
[PASS] testGetCurrentPrice_worksGivenFloorReached() (gas: 464829)
[PASS] testGetListing_returnsAllListingDetails() (gas: 474999)
[PASS] testGetListingsPaginated_returnsEmptyArrayGivenOffsetBeyondLastListing() (gas: 992163)
[PASS] testGetListingsPaginated_returnsExpectedListings() (gas: 1039943)
[PASS] testGetTotalListings_countsAllCreatedListings() (gas: 2033447)
[PASS] testListingFeeSplit_affectsETHDistributionOnCreateListing() (gas: 469979)
[PASS] testPause_revertsGivenNotOwner() (gas: 19762)
[PASS] testPause_worksGivenOwner() (gas: 46905)
[PASS] testPausedMarketplace_allowsViewFunctions() (gas: 495404)
[PASS] testPausedMarketplace_preventsCancel() (gas: 480785)
[PASS] testPausedMarketplace_preventsSettle() (gas: 623977)
[PASS] testSellerFeeOnSettledSplit_burnsAllByDefaultForThirdParty() (gas: 760834)
[PASS] testSellerFeeOnSettledSplit_routesToVaultAndOpsWhenConfigured() (gas: 884324)
[PASS] testSetAuctionDuration_affectsFutureListings() (gas: 465243)
[PASS] testSetAuctionDuration_priceDecayOverNewDuration() (gas: 513343)
[PASS] testSetAuctionDuration_revertsGivenNotOwner() (gas: 18836)
[PASS] testSetAuctionDuration_revertsGivenZeroDuration() (gas: 14014)
[PASS] testSetAuctionDuration_worksGivenOwner() (gas: 26402)
[PASS] testSetCollectionAllowed_revertsGivenNotOwner() (gas: 20439)
[PASS] testSetCollectionAllowed_revertsGivenZeroAddress() (gas: 15640)
[PASS] testSetCollectionAllowed_worksGivenOwner() (gas: 46278)
[PASS] testSetCollectionWhitelistEnabled_revertsGivenNotOwner() (gas: 18148)
[PASS] testSetCollectionWhitelistEnabled_worksGivenOwner() (gas: 38057)
[PASS] testSetDutchVaultAddress_futureFeesOnSettledGoToNewAddress() (gas: 857243)
[PASS] testSetDutchVaultAddress_revertsGivenNotOwner() (gas: 24297)
[PASS] testSetDutchVaultAddress_revertsGivenZeroAddress() (gas: 15192)
[PASS] testSetDutchVaultAddress_worksGivenOwner() (gas: 34863)
[PASS] testSetListingFeeSplitConfig_revertsGivenInvalidSplit() (gas: 14695)
[PASS] testSetListingFeeSplitConfig_revertsGivenNotOwner() (gas: 20343)
[PASS] testSetListingFeeSplitConfig_worksGivenOwner() (gas: 34818)
[PASS] testSetMaxListingFee_capsListingFeesCorrectly() (gas: 458145)
[PASS] testSetMaxListingFee_doesNotCapWhenPercentageHigher() (gas: 459032)
[PASS] testSetMaxListingFee_revertsGivenNotOwner() (gas: 18924)
[PASS] testSetMaxListingFee_revertsGivenZeroAmount() (gas: 16126)
[PASS] testSetMaxListingFee_worksGivenOwner() (gas: 29438)
[PASS] testSetOpsWallet_futureFeesOnSettledGoToNewAddress() (gas: 853783)
[PASS] testSetOpsWallet_revertsGivenNotOwner() (gas: 23483)
[PASS] testSetOpsWallet_revertsGivenZeroAddress() (gas: 14576)
[PASS] testSetOpsWallet_worksGivenOwner() (gas: 34929)
[PASS] testSetSellerFeeOnSettledSplitConfig_revertsGivenInvalidSplit() (gas: 15989)
[PASS] testSetSellerFeeOnSettledSplitConfig_revertsGivenNotOwner() (gas: 19319)
[PASS] testSetSellerFeeOnSettledSplitConfig_worksGivenOwner() (gas: 72725)
[PASS] testSetSellerFeeOnSettled_affectsFutureSettlements() (gas: 742594)
[PASS] testSetSellerFeeOnSettled_revertsGivenFeeExceeds100Percent() (gas: 13872)
[PASS] testSetSellerFeeOnSettled_revertsGivenNotOwner() (gas: 19562)
[PASS] testSetSellerFeeOnSettled_worksGivenMaxFee() (gas: 20955)
[PASS] testSetSellerFeeOnSettled_worksGivenOwner() (gas: 27249)
[PASS] testSetSellerListingFee_affectsFutureListingFees() (gas: 461397)
[PASS] testSetSellerListingFee_revertsGivenFeeExceeds100Percent() (gas: 14774)
[PASS] testSetSellerListingFee_revertsGivenNotOwner() (gas: 19738)
[PASS] testSetSellerListingFee_worksGivenMaxFee() (gas: 20889)
[PASS] testSetSellerListingFee_worksGivenOwner() (gas: 26171)
[PASS] testSettle_revertsGivenInsufficientDUTCHAllowance() (gas: 620381)
[PASS] testSettle_revertsGivenInsufficientDUTCHBalance() (gas: 673248)
[PASS] testSettle_revertsGivenListingAlreadySettled() (gas: 801649)
[PASS] testSettle_revertsGivenListingIsCancelled() (gas: 601633)
[PASS] testSettle_revertsGivenNFTTransferFails() (gas: 705555)
[PASS] testSettle_revertsGivenSlippageExceeded() (gas: 614181)
[PASS] testSettle_worksGivenProtocolListingBurnsDUTCH() (gas: 643035)
[PASS] testSettle_worksGivenValidBuyerWithSufficientDUTCH() (gas: 906210)
[PASS] testUnpause_revertsGivenNotOwner() (gas: 43846)
[PASS] testUnpause_worksGivenOwner() (gas: 36759)
[PASS] test_deployment_revertsGivenZeroDutchToken() (gas: 101660)
[PASS] test_deployment_revertsGivenZeroDutchVault() (gas: 101584)
[PASS] test_deployment_revertsGivenZeroProtocolAccount() (gas: 101785)
[PASS] test_deployment_setsAuctionDurationTo12Hours() (gas: 9003)
[PASS] test_deployment_setsCollectionWhitelistDisabled() (gas: 10797)
[PASS] test_deployment_setsDefaultListingFeeSplitTo75_25() (gas: 13082)
[PASS] test_deployment_setsDefaultSellerFeeOnSettledSplitToAllBurn() (gas: 14969)
[PASS] test_deployment_setsListingFeeTo01Eth() (gas: 10719)
[PASS] test_deployment_setsOwnerCorrectly() (gas: 13736)
[PASS] test_deployment_setsSellerFeeOnSettledTo250Bps() (gas: 10059)
```

```
[PASS] test_deployment_setsSellerListingFeeTo50Bps() (gas: 9003)
[PASS] test_deployment_setsTotalListingsToZero() (gas: 10428)
[PASS] test_deployment_startsUnpaused() (gas: 10380)
Suite result: ok. 105 passed; 0 failed; 0 skipped; finished in 33.40ms (30.86ms CPU time)

Ran 39 tests for test/AuctionAssist.t.sol:AuctionAssistTest
[PASS] test_burnProtocolShare_revertsGivenNoProtocolShare() (gas: 886541)
[PASS] test_burnProtocolShare_revertsGivenNotFunded() (gas: 493913)
[PASS] test_burnProtocolShare_worksGivenValidScenario() (gas: 1001690)
[PASS] test_claimReward_revertsGivenAlreadyClaimed() (gas: 958611)
[PASS] test_claimReward_revertsGivenNonContributor() (gas: 893471)
[PASS] test_claimReward_revertsGivenNotFunded() (gas: 495561)
[PASS] test_claimReward_worksGivenMultipleContributors() (gas: 1550088)
[PASS] test_claimReward_worksGivenSingleContributor() (gas: 953785)
[PASS] test_contribute_revertsGivenContractPaused() (gas: 60027)
[PASS] test_contribute_revertsGivenMaxContributorsExceeded() (gas: 17535738)
[PASS] test_contribute_revertsGivenNonConfiguredCollection() (gas: 2277807)
[PASS] test_contribute_revertsGivenZeroAmount() (gas: 25539)
[PASS] test_contribute_worksGiven100PercentContribution() (gas: 300312)
[PASS] test_contribute_worksGivenFullContribution() (gas: 330103)
[PASS] test_contribute_worksGivenMultipleUsers() (gas: 684686)
[PASS] test_contribute_worksGivenPartialAmount() (gas: 318697)
[PASS] test_deployment_revertsGivenZeroTokenAddress() (gas: 95687)
[PASS] test_deployment_revertsGivenZeroVaultAddress() (gas: 94584)
[PASS] test_deployment_worksGivenValidParameters() (gas: 2390362)
[PASS] test_fundRewards_revertsGivenAlreadyFunded() (gas: 921737)
[PASS] test_fundRewards_revertsGivenInvalidPurchaseId() (gas: 348701)
[PASS] test_fundRewards_revertsGivenNonOwner() (gas: 818820)
[PASS] test_fundRewards_revertsGivenZeroAmount() (gas: 489464)
[PASS] test_fundRewards_worksGivenValidParameters() (gas: 887200)
[PASS] test_getClaimableReward_returnsCorrectAmount() (gas: 882583)
[PASS] test_getClaimableReward_returnsZeroGivenAlreadyClaimed() (gas: 951189)
[PASS] test_getUserActivePositions_accumulatesGivenMultipleContributions() (gas: 409635)
[PASS] test_getUserActivePositions_returnsDetailsGivenContributions() (gas: 2927509)
[PASS] test_getUserActivePositions_returnsEmptyGivenNoContributions() (gas: 14032)
[PASS] test_getUserActivePositions_separatesGivenMultipleUsers() (gas: 523237)
[PASS] test_getUserCollections_returnsAllGivenMultipleContributions() (gas: 2914863)
[PASS] test_getUserCollections_returnsEmptyGivenNoContributions() (gas: 14609)
[PASS] test_recordPurchase_revertsGivenUnauthorizedCaller() (gas: 291440)
[PASS] test_recordPurchase_revertsGivenZeroCost() (gas: 294271)
[PASS] test_recordPurchase_worksGivenMultipleContributors() (gas: 954635)
[PASS] test_recordPurchase_worksGivenMultiplePurchases() (gas: 659256)
[PASS] test_recordPurchase_worksGivenNoContributors() (gas: 133567)
[PASS] test_recordPurchase_worksGivenOverContribution() (gas: 486816)
[PASS] test_recordPurchase_worksGivenSingleContributor() (gas: 499527)
Suite result: ok. 39 passed; 0 failed; 0 skipped; finished in 47.62ms (56.13ms CPU time)

Ran 56 tests for test/Presale.t.sol:PresaleTest
[PASS] test_checkUpkeep_isViewFunction() (gas: 2714952)
[PASS] test_checkUpkeep_returnsFalseDuringActivePresale() (gas: 2684089)
[PASS] test_checkUpkeep_returnsFalseGivenZeroSablierAddress() (gas: 2794378)
[PASS] test_checkUpkeep_returnsFalseWhenAlreadyExecuted() (gas: 2693646)
[PASS] test_checkUpkeep_returnsFalseWhenContractsNotSet() (gas: 177061)
[PASS] test_checkUpkeep_returnsFalseWithNoContributions() (gas: 2553036)
[PASS] test_checkUpkeep_returnsFalseWithOnlyBondingCurveSet() (gas: 167465)
[PASS] test_checkUpkeep_returnsTrueGivenAllConditionsIncludingSablier() (gas: 2788780)
[PASS] test_checkUpkeep_returnsTrueWhenReadyForExecution() (gas: 2704912)
[PASS] test_checkUpkeep_transitionsCorrectly() (gas: 2693603)
[PASS] test_claimVestedTokens_revertsBeforeCliffPeriod() (gas: 3551109)
[PASS] test_claimVestedTokens_revertsGivenNoStream() (gas: 3530600)
[PASS] test_claimVestedTokens_worksAfterCliffPeriod() (gas: 3613872)
[PASS] test_constructor_initializesWithValidParameters() (gas: 5986364)
[PASS] test_constructor_revertsGivenPastStartTime() (gas: 102989)
[PASS] test_constructor_revertsGivenZeroCap() (gas: 103480)
[PASS] test_constructor_revertsGivenZeroSablierAddress() (gas: 99984)
[PASS] test_createVestingStreams_calculatesAllocationCorrectly() (gas: 3527859)
[PASS] test_createVestingStreams_createsStreamsForAllContributors() (gas: 3522538)
[PASS] test_createVestingStreams_handlesCreationFailures() (gas: 3183641)
[PASS] test_createVestingStreams_setsCorrectVestingParameters() (gas: 3518693)
[PASS] test_depositEth_allocationUpdatesCorrectly() (gas: 813015)
[PASS] test_depositEth_multipleDepositsFromSameUser() (gas: 172364)
[PASS] test_depositEth_multipleUsers() (gas: 347226)
[PASS] test_depositEth_revertsAfterPresaleEnds() (gas: 35929)
[PASS] test_depositEth_revertsBeforePresaleStarts() (gas: 33420)
```

```
[PASS] test_depositEth_revertsWhenCapExceeded() (gas: 167330)
[PASS] test_depositEth_revertsWithZeroAmount() (gas: 27045)
[PASS] test_depositEth_worksAtExactCap() (gas: 152971)
[PASS] test_depositEth_worksCorrectlyDuringPresale() (gas: 164981)
[PASS] test_setContracts_addressesVisibleAfterSetting() (gas: 83631)
[PASS] test_setContracts_emitsCorrectEvent() (gas: 2537915)
[PASS] test_setContracts_enablesCheckUpkeepWhenReady() (gas: 238078)
[PASS] test_setContracts_ownerSetsValidAddresses() (gas: 2536512)
[PASS] test_setContracts_revertsWhenNotOwner() (gas: 27658)
[PASS] test_setContracts_revertsWithBothZeroAddresses() (gas: 15267)
[PASS] test_setContracts_revertsWithZeroBondingCurveAddress() (gas: 20501)
[PASS] test_setContracts_revertsWithZeroTokenAddress() (gas: 19577)
[PASS] test_setSablierV2Address_ownerCanUpdateAddress() (gas: 40439)
[PASS] test_setSablierV2Address_revertsGivenUnauthorizedCaller() (gas: 23000)
[PASS] test_setSablierV2Address_revertsGivenVestingAlreadyInitiated() (gas: 3517566)
[PASS] test_setSablierV2Address_revertsGivenZeroAddress() (gas: 13719)
[PASS] test_viewFunctions_areGasEfficient() (gas: 160213)
[PASS] test_viewFunctions_checkFundedAmountReturnsCorrectTotal() (gas: 241369)
[PASS] test_viewFunctions_getContributorCountReturnsCorrectNumber() (gas: 350722)
[PASS] test_viewFunctions_getPresaleQuoteCalculatesCorrectly() (gas: 116970)
[PASS] test_viewFunctions_getPresaleQuoteHandlesLargeAmounts() (gas: 119396)
[PASS] test_viewFunctions_getPresaleQuoteHandlesZero() (gas: 17525)
[PASS] test_viewFunctions_getUserAllocationReturnsCorrectShare() (gas: 571860)
[PASS] test_viewFunctions_getUserAllocationUpdatesWhenNewUsersJoin() (gas: 812633)
[PASS] test_viewFunctions_getUserContributionReturnsCorrectAmount() (gas: 182931)
[PASS] test_viewFunctions_returnConsistentData() (gas: 474947)
[PASS] test_viewFunctions_timeRemainingAfterPresale() (gas: 16745)
[PASS] test_viewFunctions_timeRemainingBeforePresale() (gas: 14337)
[PASS] test_viewFunctions_timeRemainingDuringPresale() (gas: 17703)
[PASS] test_viewFunctions_workBeforeContractsSet() (gas: 153790)
Suite result: ok. 56 passed; 0 failed; 0 skipped; finished in 34.55ms (42.61ms CPU time)

Ran 88 tests for test/DutchVault.t.sol:DutchVaultTest
[PASS] test_buyAndListNFT_worksGivenMultiplePurchases() (gas: 8230566)
[PASS] test_buyAndListNFT_worksGivenSuccessfulPurchase() (gas: 7680823)
[PASS] test_buyNFTForCollection_revertsGivenBalanceInsufficientForValue() (gas: 6995881)
[PASS] test_buyNFTForCollection_revertsGivenContractPaused() (gas: 6978332)
[PASS] test_buyNFTForCollection_revertsGivenInsufficientBalance() (gas: 6958479)
[PASS] test_buyNFTForCollection_revertsGivenMarketplaceFails() (gas: 7178867)
[PASS] test_buyNFTForCollection_revertsGivenNFTAlreadyOwned() (gas: 7318460)
[PASS] test_buyNFTForCollection_revertsGivenNFTNotAcquired() (gas: 7071320)
[PASS] test_buyNFTForCollection_revertsGivenNonConfiguredCollection() (gas: 6736985)
[PASS] test_buyNFTForCollection_revertsGivenPriceExceedsCap() (gas: 7000739)
[PASS] test_buyNFTForCollection_revertsGivenUnapprovedMarketplace() (gas: 6723555)
[PASS] test_cancelListing_revertsGivenContractPaused() (gas: 7654070)
[PASS] test_cancelListing_revertsGivenNFTNotListed() (gas: 7283431)
[PASS] test_cancelListing_revertsGivenNotOwner() (gas: 7635268)
[PASS] test_cancelListing_worksGivenActiveListing() (gas: 7659841)
[PASS] test_deployment_revertsGivenBothAddressesZero() (gas: 96746)
[PASS] test_deployment_revertsGivenZeroMarketplaceAddress() (gas: 98485)
[PASS] test_deployment_revertsGivenZeroTokenAddress() (gas: 98985)
[PASS] test_deployment_setsDefaultAuctionMultipliers() (gas: 4281847)
[PASS] test_deployment_worksGivenValidParameters() (gas: 4291936)
[PASS] test_getActiveCollections_returnsEmptyGivenNoCollections() (gas: 4297442)
[PASS] test_getActiveCollections_returnsOnlyActiveGivenMixed() (gas: 4559796)
[PASS] test_getBasePrice_returnsZeroGivenUnsetCollection() (gas: 4391255)
[PASS] test_getBuyIncrement_returnsZeroGivenNotSet() (gas: 4288788)
[PASS] test_getCollectionBalance_returnsZeroGivenUnconfigured() (gas: 4288788)
[PASS] test_getCollections_matchesTotalCountGivenConfigured() (gas: 4731625)
[PASS] test_getCollections_reflectsUpdatesGivenReallocation() (gas: 4563428)
[PASS] test_getCollections_returnsAllGivenConfigured() (gas: 4570386)
[PASS] test_getCollections_returnsEmptyGivenInitialState() (gas: 4296321)
[PASS] test_getInventoryFunctions_returnZeroGivenInitialState() (gas: 6524334)
[PASS] test_getMaxPriceForCollection_worksGiven100BlocksLater() (gas: 4450760)
[PASS] test_getMaxPriceForCollection_worksGivenDefaultState() (gas: 4448694)
[PASS] test_getMaxPriceForCollection_worksGivenMainnetBlock() (gas: 4449893)
[PASS] test_isListed_returnsFalseGivenUnlistedNFT() (gas: 7272196)
[PASS] test_isMarketplaceApproved_returnsFalseGivenUnapproved() (gas: 4288317)
[PASS] test_listNFTOnMarketplace_revertsGivenAlreadyListed() (gas: 7639149)
[PASS] test_listNFTOnMarketplace_revertsGivenNFTNotOwned() (gas: 6698458)
[PASS] test_listNFTOnMarketplace_worksGivenUnlistedNFT() (gas: 7673183)
[PASS] test_onListingSettled_handlesDuplicateGracefully() (gas: 7664715)
[PASS] test_onListingSettled_revertsGivenUnauthorizedCaller() (gas: 4310536)
[PASS] test_onListingSettled_worksGivenUnknownListingId() (gas: 4316406)
```

```
[PASS] test_onListingSettled_worksGivenValidListing() (gas: 7666211)
[PASS] test_onListingSettled_worksGivenVaultPaused() (gas: 7680446)
[PASS] test_receive_revertsGivenNoAllocations() (gas: 4301549)
[PASS] test_receive_worksGivenDustDistribution() (gas: 4594595)
[PASS] test_receive_worksGivenMultipleDeposits() (gas: 4726662)
[PASS] test_receive_worksGivenPrecisionHandling() (gas: 4667463)
[PASS] test_receive_worksGivenRoundRobinDustDistribution() (gas: 4810242)
[PASS] test_receive_worksGivenValidAllocations() (gas: 4688638)
[PASS] test_recordSettlement_revertsGivenAlreadyRealized() (gas: 7687729)
[PASS] test_recordSettlement_revertsGivenAutoSettlementDone() (gas: 7655648)
[PASS] test_recordSettlement_revertsGivenNFTNotListed() (gas: 7249303)
[PASS] test_recordSettlement_revertsGivenNotOwner() (gas: 7608540)
[PASS] test_recordSettlement_worksGivenLossSale() (gas: 7735288)
[PASS] test_recordSettlement_worksGivenProfitableSale() (gas: 7736905)
[PASS] test_setApprovedMarketplace_revertsGivenNonOwner() (gas: 4294591)
[PASS] test_setApprovedMarketplace_revertsGivenZeroAddress() (gas: 4284046)
[PASS] test_setApprovedMarketplace_worksGivenRemoveApproval() (gas: 4306527)
[PASS] test_setApprovedMarketplace_worksGivenValidAddress() (gas: 4319445)
[PASS] test_setAuctionAssist_revertsGivenNonOwner() (gas: 4307995)
[PASS] test_setAuctionAssist_worksGivenToggle() (gas: 4334221)
[PASS] test_setAuctionAssist_worksGivenValidAddress() (gas: 4327905)
[PASS] test_setAuctionAssist_worksGivenZeroAddress() (gas: 4312022)
[PASS] test_setAuctionMultipliers_revertsGivenNonOwner() (gas: 4290162)
[PASS] test_setAuctionMultipliers_revertsGivenUpperLessThanOrEqualLower() (gas: 4292460)
[PASS] test_setAuctionMultipliers_worksGivenOwner() (gas: 4293432)
[PASS] test_setBasePrice_revertsGivenNonConfiguredCollection() (gas: 4401918)
[PASS] test_setBasePrice_revertsGivenNonOwner() (gas: 4399095)
[PASS] test_setBasePrice_revertsGivenZeroPrice() (gas: 4394712)
[PASS] test_setBasePrice_worksGivenUpdateExisting() (gas: 4431100)
[PASS] test_setBasePrice_worksGivenValidCollection() (gas: 4505613)
[PASS] test_setBuyIncrement_revertsGivenNonConfiguredCollection() (gas: 4292160)
[PASS] test_setBuyIncrement_revertsGivenNonOwner() (gas: 4397335)
[PASS] test_setBuyIncrement_worksGivenValidCollection() (gas: 4420565)
[PASS] test_setCollectionAllocations_rebalancesGivenAllocationChange() (gas: 4607955)
[PASS] test_setCollectionAllocations_rebalancesGivenCollectionStays() (gas: 4708948)
[PASS] test_setCollectionAllocations_rebalancesGivenNoNFTs() (gas: 4543880)
[PASS] test_setCollectionAllocations_revertsGivenInvalidSum() (gas: 4324322)
[PASS] test_setCollectionAllocations_revertsGivenMismatchedLengths() (gas: 4313799)
[PASS] test_setCollectionAllocations_revertsGivenNFTsHeld() (gas: 7375910)
[PASS] test_setCollectionAllocations_revertsGivenNonOwner() (gas: 4302703)
[PASS] test_setCollectionAllocations_revertsGivenPaused() (gas: 4321084)
[PASS] test_setCollectionAllocations_revertsGivenZeroAddress() (gas: 4293963)
[PASS] test_setCollectionAllocations_worksGivenRemoveCollection() (gas: 4480310)
[PASS] test_setCollectionAllocations_worksGivenUpdateExisting() (gas: 4617057)
[PASS] test_setCollectionAllocations_worksGivenValidData() (gas: 4747882)
[PASS] test_setCollectionAllocations_worksGivenZeroBalance() (gas: 4467505)
[PASS] test_withdrawCollectionBalance_worksGivenRemovedCollection() (gas: 4544494)
Suite result: ok. 88 passed; 0 failed; 0 skipped; finished in 59.19ms (56.92ms CPU time)

Ran 72 tests for test/DUTCHToken.t.sol:DUTCHTokenTest
[PASS] testBlacklist_revertsGivenNonOwner() (gas: 19854)
[PASS] testBlacklist_revertsGivenZeroAddress() (gas: 17750)
[PASS] testBlacklist_worksGivenOwner() (gas: 47549)
[PASS] testBlacklist_worksGivenRemoval() (gas: 170326)
[PASS] testBurn_decreasesTotalSupply() (gas: 112779)
[PASS] testBurn_emitsCorrectEvents() (gas: 116581)
[PASS] testBurn_maintainsSupplyEquality() (gas: 112203)
[PASS] testBurn_revertsGivenInsufficientBalance() (gas: 101662)
[PASS] testBurn_revertsGivenUnauthorizedCaller() (gas: 103984)
[PASS] testBurn_revertsGivenZeroAddress() (gas: 18282)
[PASS] testBurn_revertsGivenZeroAmount() (gas: 101381)
[PASS] testBurn_updatesBondingCurveSupply() (gas: 113447)
[PASS] testBurn_worksGivenFuzzedAmounts(uint128) (runs: 256, : 117199, ~: 117199)
[PASS] testBurn_worksGivenMultipleBurns() (gas: 129936)
[PASS] testBurn_worksGivenValidParameters() (gas: 113725)
[PASS] testMint_emitsCorrectEvents() (gas: 105104)
[PASS] testMint_increasesTotalSupply() (gas: 101024)
[PASS] testMint_maintainsSupplyEquality() (gas: 139057)
[PASS] testMint_revertsGivenOverflow() (gas: 101890)
[PASS] testMint_revertsGivenUnauthorizedCaller() (gas: 20403)
[PASS] testMint_revertsGivenZeroAddress() (gas: 17974)
[PASS] testMint_revertsGivenZeroAmount() (gas: 19906)
[PASS] testMint_updatesBondingCurveSupply() (gas: 138081)
[PASS] testMint_worksGivenFuzzedAmounts(uint256) (runs: 256, : 107339, ~: 106741)
```

```
[PASS] testMint_worksGivenMultipleFuzzedMints(uint128,uint128) (runs: 256, : 145319, ~: 145319)
[PASS] testMint_worksGivenMultipleMints() (gas: 153834)
[PASS] testMint_worksGivenValidParameters() (gas: 111469)
[PASS] testPause_preventsBurning() (gas: 133182)
[PASS] testPause_preventsMinting() (gas: 76506)
[PASS] testPause_preventsUserBurn() (gas: 154424)
[PASS] testPause_revertsGivenNonOwner() (gas: 17907)
[PASS] testPause_worksGivenOwner() (gas: 42788)
[PASS] testTransferFrom_revertsGivenPaused() (gas: 167712)
[PASS] testTransferFrom_revertsGivenRecipientBlacklisted() (gas: 182211)
[PASS] testTransferFrom_revertsGivenSenderBlacklisted() (gas: 178301)
[PASS] testTransferFrom_worksGivenSpenderUnauthorized() (gas: 211902)
[PASS] testTransferFrom_worksGivenValidApproval() (gas: 168368)
[PASS] testTransferRestriction_allowsBondingCurve() (gas: 134119)
[PASS] testTransferRestriction_allowsP2PByDefault() (gas: 147490)
[PASS] testTransferRestriction_canBeDisabledAfterEnabling() (gas: 179225)
[PASS] testTransferRestriction_canBeEnabled() (gas: 146375)
[PASS] testTransferRestriction_revertsGivenNonOwner() (gas: 17744)
[PASS] testTransfer_revertsGivenPaused() (gas: 133443)
[PASS] testTransfer_revertsGivenRecipientBlacklisted() (gas: 145874)
[PASS] testTransfer_revertsGivenSenderBlacklisted() (gas: 144780)
[PASS] testTransfer_worksGivenValidTransfer() (gas: 155372)
[PASS] testUnpause_resumesOperations() (gas: 161957)
[PASS] testUnpause_worksGivenOwner() (gas: 34451)
[PASS] testUserBurn_createsTreasurySurplus() (gas: 159134)
[PASS] testUserBurn_doesNotAffectBondingCurveSupply() (gas: 157534)
[PASS] testUserBurn_doesNotAffectTotalSupply() (gas: 156896)
[PASS] testUserBurn_emitsCorrectEvents() (gas: 162404)
[PASS] testUserBurn_revertsGivenInsufficientBalance() (gas: 126254)
[PASS] testUserBurn_revertsGivenPaused() (gas: 154072)
[PASS] testUserBurn_revertsGivenZeroAmount() (gas: 100162)
[PASS] testUserBurn_worksGivenFuzzedAmounts(uint128) (runs: 256, : 161380, ~: 161380)
[PASS] testUserBurn_worksGivenMultipleUsers() (gas: 217089)
[PASS] testUserBurn_worksGivenValidAmount() (gas: 160729)
[PASS] test_bondingCurve_returnsCorrectAddress() (gas: 12550)
[PASS] test_deployment_emitsBondingCurveSetEvent() (gas: 1900544)
[PASS] test_deployment_emitsBondingCurveSetEventFromCalculatedAddress() (gas: 1903809)
[PASS] test_deployment_worksGivenValidParameters() (gas: 49592)
[PASS] test_isBlacklisted_returnsFalseGivenNonBlacklisted() (gas: 12594)
[PASS] test_isBlacklisted_returnsFalseGivenRemoved() (gas: 39280)
[PASS] test_isBlacklisted_returnsTrueGivenBlacklisted() (gas: 46341)
[PASS] test_setBondingCurve_revertsGivenAlreadySet() (gas: 16819)
[PASS] test_setBondingCurve_revertsGivenZeroAddress() (gas: 1847329)
[PASS] test_transfersRestricted_emitsEvent_withFalse() (gas: 35712)
[PASS] test_transfersRestricted_emitsEvent_withTrue() (gas: 46162)
[PASS] test_transfersRestricted_returnsFalseByDefault() (gas: 9038)
[PASS] test_transfersRestricted_returnsFalseGivenDisabledAgain() (gas: 33852)
[PASS] test_transfersRestricted_returnsTrueGivenEnabled() (gas: 41259)
Suite result: ok. 72 passed; 0 failed; 0 skipped; finished in 117.58ms (250.14ms CPU time)

Ran 4 tests for test/DUTCHBondingHook.t.sol:DUTCHBondingHookTest
[PASS] test_hookDeployedWithCorrectPermissions() (gas: 14386)
[PASS] test_poolInitializedCorrectly() (gas: 11195)
[FAIL: ERC20InsufficientBalance(0x29E3b139f4393aDda86303fcdAa35F60Bb7092bF, 99000000000000000000 [9.9e19],
↪  726582620450233730045 7 [7.265e21])] test_quoteDUTCHtoWETH() (gas: 773686)
[PASS] test_quoteWETHtoDUTCH() (gas: 152424)
Suite result: FAILED. 3 passed; 1 failed; 0 skipped; finished in 1.81s (2.51ms CPU time)

Ran 8 test suites in 1.81s (2.13s CPU time): 373 tests passed, 3 failed, 0 skipped (376 total tests)

Failing tests:
Encountered 1 failing test in test/DUTCHBondingHook.t.sol:DUTCHBondingHookTest
[FAIL: ERC20InsufficientBalance(0x29E3b139f4393aDda86303fcdAa35F60Bb7092bF, 99000000000000000000 [9.9e19],
↪  726582620450233730045 7 [7.265e21])] test_quoteDUTCHtoWETH() (gas: 773686)

Encountered 2 failing tests in test/Integration.t.sol:IntegrationTest
[FAIL: Rewards funded] test_journey_auctionAssistContribution() (gas: 1758076)
[FAIL: PurchaseNotFunded()] test_journey_multipleContributors() (gas: 1936531)

Encountered a total of 3 failing tests, 373 tests succeeded
```

9.3 Notes on the Test Suite

The DUTCH team delivered a relatively large test suite, including both unit tests and integration tests. Key observations of the test suite include:

- **Insufficient core business logic coverage:** Although the overall size of the test suite is considerable, coverage of key business logic such as Hooks is still insufficient. Several important business flows are not tested, leaving critical paths unverified.
- **Unstable test results:** The test suite currently contains several failing test cases, making it impossible to confirm whether the related functionalities behave as expected.

Overall, the test suite reflects an initial investment in testing infrastructure but still has some critical issues. It is recommended to supplement the missing core business logic tests and expand integration test cases to ensure that the system operates as expected in real scenarios.